Neural Networks and Principal Component Analysis

Learning from Examples without Local Minima

Pierre Baldi & Kurt Hornik | *Neural Networks*, 2(1):53–58 (1989)

DOI: 10.1016/0893-6080(89)90014-2 **red** = results **blue** = why in ML **amber** = teaching clarification

The paper in one minute

Question: If we train a neural network, might it get trapped in a bad local minimum?
The simpler model: A network with *two linear layers* compresses data, then reconstructs it.
Finding: Its best reconstruction is connected to *PCA*. In the paper's analyzed setting, the nonoptimal critical configurations are saddles, not inferior local minima.
Why we care: *Non-convex* does not automatically mean *many bad minima*.

1 1. Start with a concrete network

Picture a data point with **three features**: $x = (x_1, x_2, x_3)^T$. We want to compress those three numbers into a smaller code, and then reconstruct them. There are no nonlinear activations—only matrix multiplication.

$$\underbrace{h = Bx}_{\text{compress}}, \qquad \underbrace{\hat{x} = Ah}_{\text{reconstruct}} \implies \boxed{\hat{x} = ABx}.$$



Notation: x is the original *column vector*; B makes a shorter representation h; A maps it back to the original dimension; $\hat{x}$ ("x-hat") is the reconstruction. If $x \in \mathbb{R}^d$ and the code has $p < d$ entries, then $B \in \mathbb{R}^{p \times d}$ and $A \in \mathbb{R}^{d \times p}$.

Training objective: Make the reconstructed points as close as possible to the originals. For N samples,

$$\boxed{E(A, B) = \frac{1}{N} \sum_{i=1}^{N} \|x_i - ABx_i\|_2^2.}$$

$\|v\|_2^2 = v^T v$ is squared distance. Each term measures how much information was lost when reconstructing one data point.

2 2. Why a *linear* network has a non-convex loss

Even though $x \mapsto ABx$ is linear, we optimize the *separate* matrices A, B, whose entries multiply each other. For a one-input, one-output illustration, set $\hat{y} = ab$ and target $y = 1$:

$$L(a, b) = (1 - ab)^2 = 1 - 2ab + a^2 b^2.$$

This is *not jointly convex* in (a, b). Contrast: if we optimized one unconstrained linear map W directly, squared error would be convex in W.

Why it matters for your survey (RQ1)

Non-convex does not automatically mean *bad local minima*. It tells us the parameter-space surface is not a single convex bowl; we still have to classify its stationary points.

3 3. Stationary points, minima, and saddles — from calculus

The **gradient** contains first derivatives: $\nabla L = (\partial L/\partial a, \partial L/\partial b)^T$. A stationary point has $\nabla L = 0$; this is only a *candidate*, not proof of a minimum.

$$L(a,b) = (1-ab)^2, \qquad \frac{\partial L}{\partial a} = -2b(1-ab), \quad \frac{\partial L}{\partial b} = -2a(1-ab).$$

For both derivatives to be zero: **(i)** $ab = 1$ (loss 0: global minima), or **(ii)** $a = b = 0$ (loss 1: stationary). Is the origin a bad minimum?

$$L(\varepsilon,\varepsilon) = (1-\varepsilon^2)^2 < 1,$$
$$L(\varepsilon,-\varepsilon) = (1+\varepsilon^2)^2 > 1 \qquad (0 < \varepsilon < 1).$$

Moving in one direction *decreases* the loss; in another it *increases*. So $(0,0)$ is a **saddle point**.

3.1 3.1 The Hessian: a curvature test

The **Hessian** H collects second partial derivatives. At a stationary point, the local approximation is $L(\theta+\delta) \approx L(\theta) + \frac{1}{2}\delta^T H \delta$. Eigenvalues of H indicate curvature along its eigenvector directions.

$$H(0,0) = \begin{bmatrix} 0 & -2 \\ -2 & 0 \end{bmatrix}, \qquad \lambda_1 = 2,\ \lambda_2 = -2.$$

Positive curvature in one direction, negative in another ⇒ saddle. A **strict saddle** has a negative Hessian eigenvalue. Zero eigenvalues need further analysis: second-order information can be inconclusive.

Teaching example vs. the actual theorem

The scalar example above illustrates the logic but is *not* the paper's full classification. The authors analyze the matrix-valued squared-error objective and show its connection to covariance eigenvectors under specified assumptions.

4 4. PCA from first principles: what are u, u^T, and uu^T?

The problem: We have a point $x \in \mathbb{R}^2$, but want to store it using *one number* and then reconstruct it as well as possible. We must first choose a direction u (a column vector). Make u a **unit vector**, $u^T u = 1$, so its length does not distort the measurement. For example, $u = (1,0)^T$ chooses the horizontal axis.

Transpose means turn a column into a row. For $x = (3,2)^T$, we have $u = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $u^T = \begin{bmatrix} 1 & 0 \end{bmatrix}$. The product $u^T x$ is the **dot product**: row × column → a scalar. It extracts how far x extends along the chosen unit direction:

$$\underbrace{z = u^T x}_{\text{compress: 2 numbers}\to 1} = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 3 \\ 2 \end{bmatrix} = 3.$$

To **reconstruct**, use that scalar to scale the direction: $\hat{x} = uz = (1,0)^T 3 = (3,0)^T$. We kept the horizontal component and discarded the vertical one. Substituting $z = u^T x$ into $\hat{x} = uz$ yields $\hat{x} = u(u^T x) = (uu^T)x$.

Why uu^T is a matrix, not a number: column × row produces an *outer product*, unlike the dot product $u^T u$ (a scalar):

$$\underbrace{uu^T}_{P\text{ (projection matrix)}} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \qquad Px = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 3 \\ 2 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \end{bmatrix}.$$

The **order matters**: $u^T x$ gives one coordinate; uz turns it back into a vector; uu^T is the single matrix that performs both steps. For a different unit direction, the same construction projects onto a different line.

Which u should we choose? PCA picks the direction that loses the *least* information across all data points. For centered data, let $\Sigma = \frac{1}{N}\sum_i x_i x_i^T$ be the covariance matrix. Expanding squared reconstruction error and using $u^T u = 1$ gives

$$\underbrace{\frac{1}{N}\sum_i \|x_i - uu^T x_i\|^2}_{\text{error after compression}} = \underbrace{\operatorname{tr}(\Sigma)}_{\text{total variance}} - \underbrace{u^T \Sigma u}_{\text{variance preserved}}.$$

The first term is constant, so **minimize error ⇔ maximize** $u^T \Sigma u$ subject to $u^T u = 1$. From the eigenvalue equation $\Sigma u = \lambda u$, the maximizing direction is the eigenvector with the largest eigenvalue (PC1). With p retained dimensions, use the top p eigenvector directions.

5 5. Connect the mathematics to the paper's result

Take a centered 2D dataset with covariance

$$\Sigma = \begin{bmatrix} 9 & 0 \\ 0 & 1 \end{bmatrix}.$$

PC1 is horizontal (variance 9); PC2 is vertical (variance 1). If we preserve one direction $u(\theta) = (\cos\theta, \sin\theta)^T$, then

$$\begin{aligned} E(\theta) &= \operatorname{tr}(\Sigma) - u(\theta)^T \Sigma u(\theta) \\ &= 10 - (9\cos^2\theta + \sin^2\theta) = \boxed{1 + 8\sin^2\theta}. \end{aligned}$$

At $\theta = 0$: retain PC1; loss $= 1$ (best). **At $\theta = \pi/2$:** retain PC2; loss $= 9$ (worse). The second direction is stationary with respect to rotation, but rotating toward PC1 improves reconstruction. On the one-dimensional space of projection directions it is a maximum; in the relevant full-weight network setting, the nonoptimal full-rank configurations are saddle points.

Paper's key landscape conclusion

In the auto-associative linear-network case, covariance eigenvectors characterize the critical projection subspaces. The projector onto the **leading p-dimensional eigenspace** minimizes the reconstruction error; the paper classifies the other nondegenerate critical configurations as saddles rather than suboptimal local minima.

6 6. One subtlety: a unique solution is not unique weights

The *learned mapping* AB can be the same for many weights. For any invertible hidden-space matrix C,

$$\boxed{AB = (AC)(C^{-1}B)}.$$

Thus even if the optimal *projection* is unique when the relevant eigenvalues are distinct, there are many equivalent (A, B) pairs. This parameter symmetry creates flat directions and helps explain why neural-network Hessians may have zero eigenvalues.

7 7. What the result does *not* say

- **Architecture:** it concerns *linear* networks; the same "no poor local minima" claim does not extend automatically to nonlinear ReLU networks.
- **Loss/data:** the central PCA characterization depends on quadratic error and the paper's rank/eigenvalue hypotheses; non-distinct eigenvalues and rank-deficient cases require care.
- **Optimization:** describing all critical points is not the same as proving that a particular GD/SGD run escapes saddles quickly or converges globally.

8 8. Connection to our survey and next paper

Takeaway — Geometry (RQ1), limits (RQ4)

$$\boxed{\nabla L(\theta^*) = 0 \;\not\Rightarrow\; \theta^* \text{ is a minimum.}}$$

A *non-convex* problem can still possess a favorable landscape: global minima and saddles, but no poor local minima in the analyzed regime. **Baldi–Hornik tells us what critical points look like.** Saxe et al. (2014) is the natural next reading for *how gradient-based training moves* through a deep linear network over time.

Reference: Baldi, P., & Hornik, K. (1989). *Neural Networks and Principal Component Analysis: Learning from Examples without Local Minima. Neural Networks*, 2(1), 53–58. doi:10.1016/0893-6080(89)90014-2. *Worked scalar and 2×2 examples are explanatory additions, not direct reproductions of the authors' proofs.*